WearWise: An Intelligent Knowledge-Based System for Sustainable Wardrobe Management

1st Pau Porta Roca
Dept. of Engineering
La Salle - Universitat Ramon Llull
Barcelona, Spain

2nd Giovanni Luca Giraldo Cabana
Dept. of Engineering
La Salle - Universitat Ramon Llull
Barcelona, Spain

3rd Francesc Mateu Vallespín
Dept. of Engineering
La Salle - Universitat Ramon Llull
Barcelona, Spain

4th Jan Francesc Porcar
Dept. of Engineering
La Salle - Universitat Ramon Llull
Barcelona, Spain

***Abstract*—The rapid expansion of fast fashion has resulted in massive clothing overproduction and growing environmental impact. This controversial issue is highly affected by human wardrobe rotation, the struggle to effectively combine garments, leading to repeatedly wear the same outfits while leaving most clothes unused. This underutilization, apart from wearing out clothes, triggers a constant feeling of scarsity that leads to go shopping again.**

The majority of the current recommendation systems are tailored to online retail sites and primarily aim to increase sales through applications of data-driven methods like collaborative filtering or deep learning. On the other hand, personal wardrobe management remains almost entirely unsupported, particularly in the areas of transparent decision-making and consideration of real-life limitations such as weather, dress codes, or social contexts.

In the present study, we introduce a prototype knowledge-based system for outfit recommendations aimed at enhancing the use of the clothes already owned rather than pushing for new purchases. The system is capable of digitizing clothing and represent them as structured knowledge. A forward-chaining inference engine then draws conclusions about this knowledge through an explicit set of fashion rules to produce recommendations for outfits that are both context-aware and explainable. This work, by prioritizing logical reasoning, constraint enforcement, and transparency, demonstrates how knowledge-based systems can assist in making more sustainable and responsible daily decisions.

***Index Terms*—Sustainable fashion, wardrobe management, knowledge-based systems, explainable artificial intelligence, outfit recommendation, rule-based reasoning, context-aware recommendation systems.**

I. Introduction

The proliferation of "Fast Fashion" has fundamentally altered consumer behavior, leading to a rapid accumulation of garments. However, this abundance has not translated into satisfaction. This is an instance of the paradox of choice, where an increase in options leads to decision paralysis. In the context of daily dressing, this manifests as the feeling of having nothing to wear despite a full closet.

Beyond the psychological toll, the environmental impact is severe. The Ellen MacArthur Foundation reports that the average number of times a garment is worn has decreased by 36% in the last 15 years [14]. Extending the life of existing clothes is critical in order to reduce carbon waste, and water footprints.

To tackle this psychological and environmental challenge, we propose *WearWise*. This system is not merely a digital inventory; it is a **Knowledge-Based System (KBS)** designed to emulate the reasoning of a human stylist.

The engineering challenge in building such a system is twofold:

1) **The Perception Gap:** Converting raw pixels into a structured, usable format a computer can work with.
2) **The Logic Gap:** Once the system has the facts it needs detailed conditional logic to determine if a garment fits a specific situation.

This paper details the engineering journey to bridge these gaps. We present an architecture that employs

Large Language Models (LLMs) for vision and explicit programming for reasoning. The remainder of this paper is organized as follows: Section II discusses the theoretical framework. Section III details the Data Engineering and Persistence layer. Section IV analyzes the AI extraction pipeline. Section V covers the Inference Engine, and Section VI details the Frontend and Results.

II. Theoretical Framework

The design of WearWise is informed by a critical analysis of existing recommendation methodologies. Table I summarizes the gap analysis performed.

A. Collaborative Filtering (CF)

The industry standard in most commercial platforms (e.g., Amazon, Netflix) is Collaborative Filtering. This method relies on the fact that if User A and User B share purchase history, they will like similar items [10].

- **Advantage:** Highly scalable and effective for sales.
- **Limitation:** It suffers from the "Cold Start" problem. More importantly, it ignores the *content* of the item. It recommends what is popular, not necessarily what is aesthetically or climatologically appropriate for a specific scenario.

B. Content-Based Deep Learning

Recent advances in Computer Vision use Convolutional Neural Networks (CNNs) to extract features (embeddings) from images and calculate vector similarity [9].

- **Advantage:** Can find visually similar items.
- **Limitation:** These models are "Black Boxes." They offer no explainability. If a neural network recommends a swimsuit for a snowy day, it is difficult to debug or constrain.

C. Rule-Based Expert Systems

Classical Expert Systems (ES) use an inference engine and a knowledge base of facts and rules [1].

- **Advantage:** They are deterministic, verifiable, and explainable. We can program a strict rule: $IF\ Temperature < 10\ THEN\ Exclude\ Shorts$.
- **Limitation:** The Knowledge Acquisition "Bottleneck." Historically, ES failed because they required users to manually input hundreds of facts.

D. The WearWise Approach:

WearWise adopts an alternative architecture, an approach that seeks to combine the best of both worlds: LLMs are used to extract raw data while the reasoning is all encoded through RULE - FACT relations. Human expertise is encoded into explicit rules used through an inference engine to extract explainable and traceable conclusions. In our implementation:

- **AI Component:** GPT-4o acts as the perception layer, handling fuzzy, unstructured input (images).
- **Inference Component:** A Python-based Forward Chaining engine handles the structured reasoning.

To justify the selection of this approach, Table I provides a comparison between the proposed architecture and traditional methodologies.

TABLE I: Comparison of Recommendation Approaches

Feature	Collab. Filtering	Pure CNN	WearWise
Explainability	Low	Very Low	**High**
Constraint Enforcement	None	Weak	**Strict**
Cold Start Handling	Poor	Good	**Excellent**
Context Awareness	Low	Medium	**High**

III. Data Engineering and Persistence

A robust data layer is the foundation of any KBS. Given the disperse nature of fashion items, selecting the right representation strategy was crucial.

A. Database Selection: MongoDB (NoSQL)

We selected **MongoDB Atlas** as our persistence layer. Unlike Relational Database Management Systems (RDBMS) like PostgreSQL, which require rigid schemas, MongoDB is document-oriented [17]. This is essential for fashion because attributes vary by category:

- A **Shoe** requires attributes like `heel_height` and `sole_type`.
- A **Jacket** requires `insulation_level` and `hood_type`.

In SQL, this would require complex Entity-Attribute-Value (EAV) tables or sparse columns. In MongoDB, we simply store the JSON document containing garment data as provided in the extraction phase.

For clarity and readability, simplified versions of some functions are shown in the following listings. The production code includes additional validation and error handling logic that has been omitted here. This decision avoids overwhelming the reader while preserving the core logic of the system.

```python
#initialize the MongoDB client
client = MongoClient(os.getenv("MONGODB_URI"))

#specific database and colleciton
db = client[os.getenv("MONGODB_DB")]
collection = db[os.getenv("MONGODB_COLLECTION")]

#Inserts a single document into the configured MongoDB collection:
def save_document(doc):
    collection.insert_one(doc)

def get_database(url, db_name):
    client = MongoClient(url)
    return client[db_name]

def get_collection(db, collection_name):
    return db[collection_name]

def load_garments(collection):
    # Retrieve full documents to access _id and specific attributes
    cursor = collection.find({})
    return list(cursor)
```

Listing 1: Database Connection (db.py)

B. Secure Database Connection Implementation

To ensure security and scalability, we implemented a single pattern for the database connection in the module `db.py`. This prevents the creation of multiple unnecessary connections to the Atlas cluster, which could lead to timeouts or memory leaks. We utilize `python-dotenv` to load credentials securely.

```python
import os
from pymongo import MongoClient
from dotenv import load_dotenv

# Load environment variables from .env file
load_dotenv()

def get_database():
    """
    Establishes a singleton connection to MongoDB Atlas.
    Returns the database object.
    """
    CONNECTION_STRING = os.getenv("MONGO_URI")
    try:
        client = MongoClient(CONNECTION_STRING)
        # Verify connection
        client.admin.command('ping')
        return client['wearwise_db']
    except Exception as e:
        print(f"Error connecting to MongoDB: {e}")
        return None

# Collection references
db = get_database()
wardrobe_collection = db['wardrobe']
```

Listing 2: Secure Database Connection (db.py)

C. Data Ontology and Schema

Although MongoDB is schema-less, the Inference Engine requires a predictable structure. We defined a strict ontology in `normalization.json`. The application enforces the following document structure before insertion:

```json
{
  "_id": "uuid_v4_string",
  "user_id": "demo_user",
  "filename": "winter_coat_01.jpg",
  "normalized_attributes": {
      "category": "outerwear",
      "sub_category": "coat",
      "color": "camel",
      "pattern": "solid",
      "material": "wool",
      "season": ["winter", "autumn"],
      "formality": "business_casual"
  },
  "raw_attributes": { ... },
  "image_base64": "data:image/jpeg;base64/...",
  "created_at": "2025-01-20T10:00:00Z"
}
```

Listing 3: Garment Document Schema in MongoDB

D. Image Storage Strategy

Standard practice often involves storing images in object storage (e.g., AWS S3) and keeping references in the DB. However, to simplify the architecture for this prototype and reduce latency during the rendering of the wardrobe grid, we implemented a Base64 encoding strategy.

E. Optimization: Pre-processing

Before encoding, it is crucial to optimize images to avoid database bloat. We implemented a pre-processing using the `Pillow` library in Python to resize and encode images efficiently.

```python
def to_thumbnail_base64(image_bytes, max_size=512):
    image = Image.open(io.BytesIO(image_bytes)).convert("RGB")           #reads and converts to RGB
    image.thumbnail((max_size, max_size))                        #resize maintaingn ratio (512 pixels max)
    buf = io.BytesIO()
    image.save(buf, format="JPEG", optimize=True, quality=85)            #asve to buffer instead of hard drive

    return base64.b64encode(buf.getvalue()).decode("utf-8")              #convert to text string
```

Listing 4: Image Optimization Logic (UserInterface.py)

This allows the Streamlit frontend to display images immediately using the response data, eliminating the need for extra server requests and significantly speeding up the interface.

F. Global System Architecture

To provide a holistic understanding of the WearWise platform, Fig. 1 presents the complete inner processes of the program.

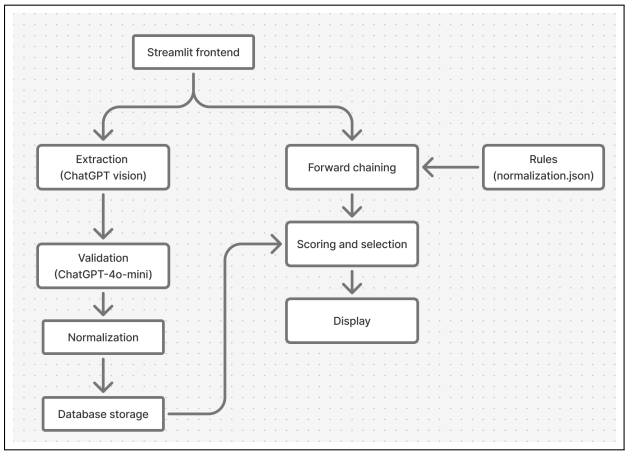


Fig. 1: End-to-end architecture of the WearWise system, illustrating the interaction between the user interface, AI extraction layer, normalization pipeline, symbolic inference engine, and MongoDB persistence layer.

The architecture is deliberately modular. Each subsystem can evolve independently without breaking the overall system. This separation follows classical software engineering principles such as *low coupling* and *high cohesion* [16], ensuring long-term maintainability and scalability.

IV. The Subsystem: AI Extraction

The main thing that WearWise does is that it automates the process of analyzing data. The way this part of WearWise works is controlled by the code in the agent.py module. This module is in charge of making calls to the intelligence provider that WearWise uses, relying on this module to make everything work smoothly.

A. Model Selection: Why OpenAI GPT-4o?

We chose a Large Language Model from OpenAI after we looked at options like Google Gemini 1.5 Pro and Anthropic Claude 3.5 Sonnet. The reason we picked the Large Language Model from OpenAI is because it was easier to use in our project not because it's the best model out there. We thought about what would work for us and the Large Language Model from OpenAI was the best choice, for what we needed to do.

The GPT-4o model was chosen for its reliability in following JSON schema rules, which minimized errors during data extraction. Its unique training on audio, vision, and text simultaneously allows it to process images faster than models that use separate stages. Additionally, our tests showed it is highly accurate at identifying specific fashion details, such as distinguishing between closely related but similar garments, without requiring extensive examples.

1) **Native JSON Mode:** The selected model showed more consistent adherence to rigid JSON schemas during testing, which reduced parsing errors in the extractor pipeline.
2) **Latency:** Being an "omni" model trained natively across audio, vision, and text, GPT-4o demonstrates lower latency for image analysis than pipeline-based models.
3) **Zero-Shot Reasoning:** In our internal benchmarks, GPT-4o showed higher accuracy in discerning subtle fashion attributes without requiring many examples [7].

During preliminary testing, the selected model showed more reliable adherence to the required JSON schema and acceptable latency for image-based attribute extraction. Other evaluated models occasionally produced malformed outputs or required additional post-processing. Given the limited scope of the project and the need for stable integration, this trade-off was considered acceptable.

B. Zero-Shot Prompt Engineering

We use a zero-shot approach, which means we do not give it any examples to work with. This helps to keep things easy to set up. The way we design the prompt is not about getting the most accurate information out of it. Instead we want the computer to give us information that is consistent and organized so it is easy to work with later on. We want this information to be in a format that can be understood, used, an easily manipulated.

The instructions have three parts. First they state what to do, which is to simulate a fashion expert. Then the prompt includes negative constraints, directing the agent to ignore background noise and avoid inferring details for visually ambiguous or hidden garments. Finally, the system imposes a strict output constraint, mandating that the model return a syntactically valid JSON object. These formatting directives act as rigid guardrails, ensuring

the extracted data adheres to the required schema. Furthermore, assigning the specific persona of a 'technical fashion expert' primes the model to prioritize domain accuracy and terminological precision throughout the task.

The default temperature is 0.1, and when it comes to language models a high temperature makes the language more creative. For this reason, and attempting to get maximum precision in the outputs this variable is left untouched.

Regarding the 3 layers of instructions:

1) **Role Definition:** "You are a technical fashion expert."
2) **Constraint Injection:** "Ignore background items,"
3) **Format Enforcement:** "Return ONLY a valid JSON object."

```python
def extract_raw_attributes(image_bytes):
    """Call OpenAI Vision to extract structured attributes for all garments."""
    # Encode image bytes to base64 for the chat vision payload
    image_b64 = base64.b64encode(image_bytes).decode("utf-8")
    response = client.chat.completions.create(
        model="gpt-4o",
        messages=[
            {"role": "system", "content": BASE_PROMPT},
            {"role": "user", "content": [
                {"type": "text", "text": "Extract attributes for every visible garment piece."},
                {
                    "type": "image_url",
                    "image_url": {"url": f"{image_b64}"}
                }
            ]}
        ],
        response_format={"type": "json_object"}
    )

    message = response.choices[0].message
    # Extract the JSON text from the content blocks
    if isinstance(message.content, str):
        text = message.content.strip()
    else:
        blocks = message.content or []
        parts = [block.text for block in blocks if getattr(block, "text", None)]
        text = "".join(parts).strip()

    if not text:
        raise ValueError("No text content returned from the model.")

    return json.loads(text)
```

Listing 5: Python Implementation of the Vision Agent (agent.py)

C. The Normalization Pipeline

Another problem can arise from using AI as a data extractor, inconsistency. Different executions on the same image can yield slightly varied outputs. This poses a serious threat to the stability of the Inference Engine which depends on data uniformity to work as expected.

To solve this, we implemented a deterministic Normalization Layer (`normalizer.py`). This module acts as an intermediate step between the AI Agent and the Database. It loads the `normalization.json` ontology into memory.

```python
with open(os.getenv("NORMALIZATION_CONFIG_PATH"), "r") as f:
    CONFIG = json.load(f)["normalization"]

def normalize_garment(garment):

    "Normalize attributes for a single garment."

    raw_type = garment.get("garment_type", "").lower().strip()

    category = CONFIG["categories"].get(raw_type, raw_type)

    #returns the normalizaed garment dictionary
    return {
        "garment_type": category,
        "specific_type": raw_type,
        "sex": garment.get("sex"),
        "color": normalize_value(garment.get("color"), CONFIG["colors"]),
        "pattern": normalize_value(garment.get("pattern"), CONFIG["patterns"]),
        "season": normalize_value(garment.get("season"), CONFIG["seasons"]),
        "material": normalize_value(garment.get("material"), CONFIG["materials"]),
        "style": normalize_value(garment.get("style"), CONFIG["styles"]),
        "formality": normalize_value(garment.get("formality"), CONFIG["formality"]),
        "occasion": normalize_value(garment.get("occasion"), CONFIG["occasions"]),
    }
```

Listing 6: Normalization Logic (normalizer.py)

This step ensures that the database remains clean and format consistent, satisfying the strict requirements of the inference engine.

```python
def normalize_value(value, dictionary):

    "Normalizes an attribute with the corresponding dictionary in normalizatino"

    if not value:
        return None
    key = value.strip().lower()
    return dictionary.get(key, value)
```

Listing 7: Deterministic Normalization (normalizer.py)

V. The Symbolic Subsystem: Inference Engine

The deterministic core of WearWise is implemented in the `forward_chaining.py` module. While the AI component deals with probabilities, this engine deals with certainties. It implements a classical Forward Chaining algorithm, chosen for its data-driven nature: it constructs a solution (an outfit) from the available facts (wardrobe + context).

Unlike Large Language Models (LLMs), which are inherently stochastic and may 'hallucinate' or suggest unsafe combinations based on statistical likelihood, the rule-based engine is strictly constrained by a predefined knowledge base in normalization.json. This ensures that if the temperature fact is set to 5°C, the system will always trigger the need:coat and forbid:shorts rules without exception. By separating the reasoning from the perception, we ensure that the final recommendation is not just a 'guess' based on a pattern, but a verifiable conclusion derived from explicit fashion and safety axioms.

```
"formality_rules": {
      "formal": { "allowed":["shirt","blazer","
    ↪ dress_pants","formal_shoes"], "forbidden":["
    ↪ tshirt","jeans","sneakers", "shorts", "
    ↪ sandals", "leggings"] },
      "business_casual": { "allowed":["shirt","polo"
    ↪ ,"chinos","neutral_sneakers"] },
      "casual": { "allowed":["tshirt","jeans","
    ↪ sneakers"] }
```

Listing 8: Formality rules (normalization.json)

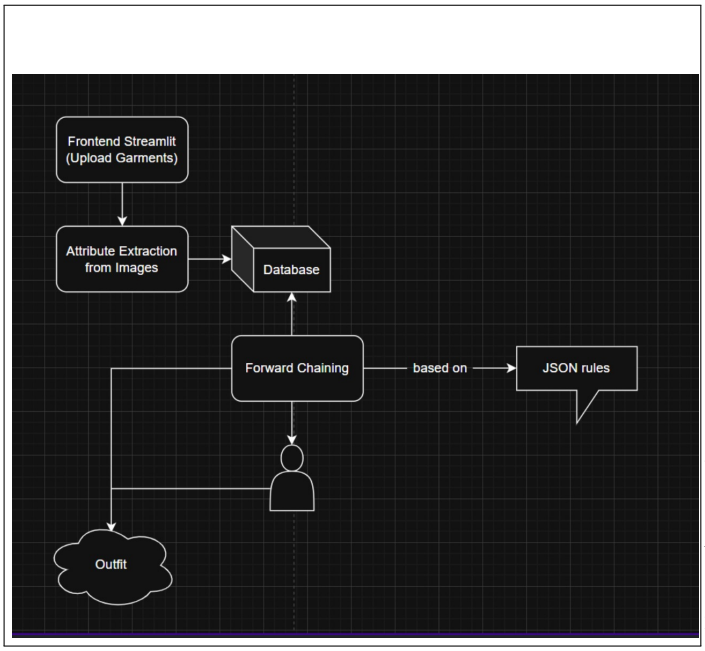


Fig. 2: Overall architecture structure.

The process begins by transforming user context, such as current temperature, season, and desired formality, into a set of logical facts via the `build_user_facts` function. These initial facts populate the set the system will use later. The engine then enters an iterative loop, continuously scanning the rule definitions in normalization.json to identify conditions satisfied by the current fact set. When a rule's conditions are met; for instance, a temperature below 5°C, the engine 'fires' and updates with new derived facts, such as specific garment requirements (need:coat) or physical constraints (forbid:shorts). This cycle repeats until a point is reached, the state where no additional rules can be triggered. By ensuring all applicable rules are exhaustively applied before moving to the selection phase, the engine guarantees that every recommendation is consistent with the established fashion and safety axioms of the knowledge base.

A. Formal Algorithmic Representation

To guarantee the determinism of our system, we formalize the Forward Chaining process in Algorithm 1. This ensures that the state transition from initial context to final recommendation is traceable and reproducible.

Algorithm 1 WearWise Forward Chaining Logic

Input: UserContext C, WardrobeSet W, RuleBase R
Output: RankedRecommendationList L
$Facts \leftarrow$ GenerateInitialFacts(C)
$NewFactsFound \leftarrow$ True
while $NewFactsFound$ **do**
  $NewFactsFound \leftarrow$ False
  for all $rule \in R$ **do**
    if $rule.condition \subseteq Facts$ **and** $rule.conclusion \notin Facts$ **then**
      $Facts \leftarrow Facts \cup rule.conclusion$
      $NewFactsFound \leftarrow$ True
    end if
  end for
end while
$Candidates \leftarrow$ Filter$(W, Facts)$
$L \leftarrow$ HeuristicRanking$(Candidates, Facts)$
return L

B. Knowledge Representation

Rules are not hardcoded as simple `if-else` statements scattered throughout the code. Instead, they are represented as objects or data structures that can be iterated over. This allows for cleaner separation of logic

and data, facilitating future extensions. The forbidden, required and allowed structures are particular examples of rule encoding.

```
"temperature_rules": {
      "rules": [
        { "name":"very_cold","min":-50,"max":5,"required":["puffer jacket","sweater","scarf","gloves","boots"], "forbidden":["shorts","tank_top","tshirt"] },
        { "name":"cold","min":6,"max":12,"required":["jacket","light_sweater"],"forbidden":["shorts", "tank_top", "tshirt"] },
        { "name":"mild","min":13,"max":18,"required":["shirt","thin_sweater"] },
        { "name":"warm","min":19,"max":26,"required":["tshirt","light_pants"], "forbidden":["puffer jacket","coat"] },
        { "name":"hot","min":27,"max":60,"required":["tank_top","shorts","sandals"], "forbidden":["outerwear"] }
      ]
```

Listing 9: Rule Representation Structure

C. The Inference Algorithm

The engine operates on a cyclic "Match-Resolve-Act" pattern. It maintains a Working Memory of facts.

- Initial Facts: User Context (e.g., $Temp = 4°C$, $Occasion = Work$).
- Derived Facts: Needs inferred by rules (e.g., $Need = Coat$).

The Python implementation follows this logic flow:

```
def forward_chain(facts, wardrobe_rules):
    """
    Given the intinal set of facts from the user derives new facts with the wardrobe rules.
    """
    rules = wardrobe_rules["rules"]
    changed = True
    while changed:
        changed = False

        #temperature rules
        temp_fact = next((f for f in facts if f.startswith("temperature:")), None)
        if temp_fact:
            temp_val = float(temp_fact.split(":", 1)[1])
            for tr in rules["temperature_rules"]["rules"]:
                if tr["min"] <= temp_val <= tr["max"]:
                    band_fact = f"temp_band:{tr['name']}"
                    if band_fact not in facts:
                        facts.add(band_fact); changed = True
                    for req in tr.get("required", []):
                        rf = f"need:{req}"
                        if rf not in facts: facts.add(rf); changed = True
                    for forb in tr.get("forbidden", []):
                        ff = f"forbid:{forb}"
                        if ff not in facts: facts.add(ff); changed = True
                        break
```

Listing 10: Forward Chaining Logic Snippet (Temperature module)

D. Formal Definition of the Inference Process

To complement the implementation-level description, this subsection provides a formal and abstract representation of the inference mechanism used in WearWise.

Algorithm 2 WearWise Forward Chaining Inference

```
Initialize WorkingMemory with ContextFacts
InferredFacts ← ∅
repeat
    for all Rule R_i in RuleBase do
        if Antecedents of R_i ⊆ WorkingMemory then
            Add Consequents of R_i to InferredFacts
        end if
    end for
    WorkingMemory ← WorkingMemory ∪ InferredFacts
until No new facts are inferred
Filter wardrobe based on WorkingMemory
Rank valid garments using heuristic scoring
```

E. Sample Rule Set

Table II shows a representative subset of the rule base implemented in the system.

TABLE II: Example Inference Rules

Rule ID	Condition	Inference
R1	Temp $\leq 5°C$	`need_coat`
R2	Temp $\leq 5°C$	`forbid_shorts`
R3	Occasion = Work	`forbid_sport`
R4	Temp $\geq 25°C$	`forbid_puffer_jacket`

F. Heuristic Scoring System

A purely boolean rule system ($Valid/Invalid$) is insufficient for fashion, where "matching" is a gradient. We implemented a Heuristic Scoring System to rank valid candidates.

The score S for a garment g is calculated as:

$$S(g) = S_{\text{base}} + \sum(w_i \cdot M_i) - \sum(p_j \cdot V_j) \quad (1)$$

Where:

- M_i (Matches): Weighted rewards for aesthetic compatibility (e.g., **+10** for exact occasion match, **+5** for formality alignment, **+3** for color harmony).

- V_j (Violations): Penalties for sub-optimal fits (e.g., **-5** for occasion mismatch, or a **-20** fallback penalty if the item fails strict social criteria).

This mathematical approach ensures that while a hoodie *could* technically be worn to a dinner (it is physically safe), a blazer will always outrank it due to the higher formality weight ($+5$) and occasion alignment ($+10$).

The candidate selection process operates in two distinct phases:

1) **Hard Filtering (Safety):** Before scoring, the `is_garment_safe` function acts as a boolean gatekeeper. It eliminates items that violate physical constraints (e.g., thermal inadequacy) defined in the knowledge base. These items are removed from the search space entirely ($S = -\infty$).
2) **Heuristic Scoring:** surviving candidates are evaluated using the scoring function. Here, the `is_garment_strict_match` function differentiates between ideal candidates and fallback options.

G. Conflict Resolution Strategy

When multiple rules are simultaneously applicable, WearWise employs a hierarchical constraint satisfaction strategy. Rules associated with safety and climatological constraints (e.g., extreme cold) function as pre-conditions, evaluated strictly before aesthetic preferences. This architectural decision ensures that hard constraints are never compromised, even if soft stylistic rules would otherwise maximize the score for an unsafe garment.

VI. Frontend Implementation

The user interface is built using Streamlit, a Python-native framework. This choice was strategic: it allows the Knowledge-Based System (KBS) components to live in the same codebase as the UI, reducing latency and architectural complexity.

As observed in Fig. 3, the application interface provides a clear environment to manage the database and trigger the Inference Engine. By developing this unified practical prototype, we facilitate immediate user feedback and verification based on real implementation, addressing a fundamental issue often neglected in recommendation schemes. The integrated design seen in Fig. 3 effectively handles the scalability problem of wardrobe digitization while ensuring the system provides logic, constraints, and explanations rather than mere predictions.

A. Unified Workflow and Logic

The interface allows users to perform the entire styling lifecycle in one view:

1) **Image input:** Users upload garments via the file uploader to update the database.
2) **Contextual Fact Base:** Reactive sliders and selectboxes allow users to input situational data.
3) **Explainable Recommendation:** The Inference Engine processes the knowledge against the current facts and uses the virtual wardrobe to generate logical outfit recommendations.

B. Design and Management

A significant design challenge in web-based inference is data persistence; typically, variables are lost between re-runs. However, the system serializes extracted attributes and images directly to the database. This ensures that the user's wardrobe state is preserved across different sessions and interface reloads, allowing for immediate retrieval during the inference phase.

```
for file in uploaded_files:
            image_bytes = file.read()

            with st.status(f"Processing {file.name
    ↪ }...", expanded=True) as status:

                [...]

                thumb_b64 = to_thumbnail_base64(
    ↪ image_bytes)
                raw_garments = result["raw"].get("
    ↪ garments", [])
                corrected_garments = result["
    ↪ corrected"].get("garments", [])
                normalized_garments = result["
    ↪ normalized"].get("garments", [])

                for idx, norm in enumerate(
    ↪ normalized_garments):
                    stored_doc = {
                        "_id": str(uuid.uuid4()),
                        "filename": file.name,
                        "garment_index": idx,
                        "normalized_attributes":
    ↪ norm,
                        "corrected_attributes":
    ↪ corrected_garments[idx] if idx < len(
    ↪ corrected_garments) else None,
                        "raw_attributes":
    ↪ raw_garments[idx] if idx < len(raw_garments)
    ↪ else None,
                        "image_base64": thumb_b64
                    }
                    save_document(stored_doc)
```

Listing 11: Image upload code snippet (UserInterface.py)

VII. Results and Validation

To validate the system, we populated the database with a set of 38 garments of diverse style and fashion.

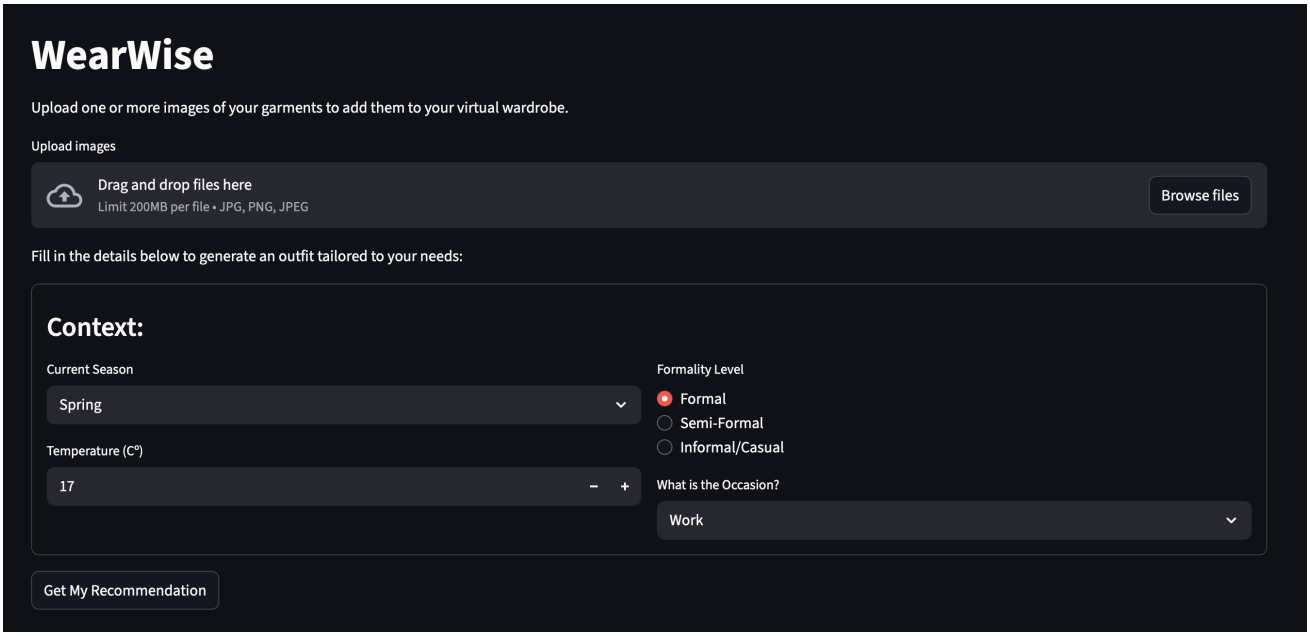


Fig. 3: WearWise Dashboard: The top section handles garment upload (MongoDB database), while the lower part allows the user to input context parameters and run the engine.

A. Scenario A: The "Winter Commute"

Context: Temp: 0°C, Occasion: Work.

- **Inference Trace:** The engine triggered the rule `very_cold`. This activated both required and forbidden sets, in particular: `required:puffer_jacket`.
- **Filtering:** Items with attribute `occasion:sport` received a score penalty.
- **Ranking:** A "Sweater" received +20 points (+10 Temp, +10 Formality).
- **Result:** The system successfully recommended the puffer jacket, paired with boots and trousers.

B. Scenario B: The "Summer Party"

Context: Temp: 28°C, Occasion: Party.

- **Inference Trace:** The engine triggered `rule_hot`.
- **Filtering:** "Heavy" items (Coats, Hoodies) were filtered out.
- **Ranking:** A "shirt" scored highest due to the `season:summer` match.
- **Result:** Correct recommendation of lightweight, formal attire.

C. Explainability Analysis

A key advantage of symbolic reasoning is explainability. For every recommendation, WearWise produces a trace of activated rules and applied heuristics.

For example, in Scenario A, the explanation generated was:

- Context fact `very_cold` activated rule R1.
- Rule R1 inferred `required_puffer_jacket`.
- Rule R2 excluded summer garments.
- Heuristic scoring favored wool-based materials.

This transparency is critical in gaining user trust, especially when compared to opaque recommendation engines.

D. Future Extensions

Future work includes:

- Integration of real-time weather APIs.
- Cultural adaptation rules (e.g., dress codes by country).
- Extended color harmony logic.
- Multi-outfit planning (weekly recommendations).
- Deploying the program to the cloud using a virtual machine (instead of executing locally).

VIII. Discussion

The development of WearWise highlighted the viability of systems for consumer applications. The Agent effectively handled the data uploading in a clear and understandable way. Meanwhile, the engine ensured that

the advice remained safe and logical, avoiding the hallucinations common in pure LLM approaches.

However, limitations exist. The system's understanding of color harmony is based on rigid graph rules (e.g., "Blue matches White"). It lacks the nuance to understand complex pattern clashing (e.g., plaid vs. polka dots), which a visual neural network might detect better.

A. Comparison with Existing Approaches

Compared to purely neural recommendation systems, WearWise prioritizes logical consistency and user trust over predictive novelty. While neural models may capture subtle visual trends, they lack constraint enforcement, which can result in contextually inappropriate recommendations.

Conversely, classical expert systems guarantee logical correctness, but traditionally fail due to manual knowledge acquisition. The prototype bridges this gap by automating fact acquisition through AI, achieving a balance between flexibility and control.

B. Ethical and Sustainability Considerations

By encouraging reuse of existing garments instead of promoting new purchases, WearWise aligns with sustainable consumption principles. Unlike commercial fashion recommendation systems, the platform is intentionally non-commercial, avoiding bias toward sponsored products.

C. Limitations

This project was developed under academic time constraints and evaluated using a relatively small wardrobe size. As a consequence, the results should be interpreted as indicative rather than definitive. Additionally, several design decisions favored implementation simplicity and system transparency over optimal performance.

IX. Conclusion

The development and validation of *WearWise* perfectly portrays the difference between purely AI approaches and Knowledge Based Systems, and how the latter is capable of producing explainable and traceable results as a result of explicit knowledge definition. Parallelly, this project was driven by a dual motivation: addressing the psychological burden of the choice fatigue in daily dressing and mitigating the environmental impact of textile waste.

From a **user-centric and societal perspective**, WearWise successfully shifts the paradigm of fashion recommendation. Unlike commercial algorithms optimized to trigger new purchases, our system is designed to maximize the utility of the user's existing inventory. By providing a digital stylist that is aware of the user's context, the system effectively offloads the cognitive burden of decision making. This encourages a "shop your own closet" philosophy, promoting the circular economy and extending the lifecycle of garments. The transparency of the rule-based recommendations fosters user trust, as the system explains why an outfit was chosen (e.g., selected wool due to low temperatures), contrasting sharply with the opaque nature of black-box neural networks.

From a **technical and engineering perspective**, this work validates the efficacy of Knowledge-Based architectures in unstructured domains. We have demonstrated that the limitations of Expert Systems can be effectively solved using modern Vision Models as autonomous data ingestion agents. By using rules and facts, we achieved a system that is both flexible in handling visual data and rigorous in enforcing logical constraints. The modular design, which leverages databases and Python for logic processing, ensures that the system is scalable and maintainable.

Future work will focus on enhancing the system's autonomy by integrating real-time API services for weather and calendar events, further reducing manual input. Additionally, we aim to expand the ontology to incorporate cultural dress codes and further color logic, allowing for more varied outfits to be put together.

In summary, WearWise proves that explainable logic driven AI is not only a viable alternative to deep learning but a safer approach requiring strict adherence to constraints, safety, and interpretability.

Despite its limitations, the project demonstrates that combining explicit reasoning with modern AI tools is a practical and educationally valuable approach for real-world problem solving.

References

[1] J. Giarratano and G. Riley, *Expert Systems: Principles and Programming*. Thomson Course Technology, 4th ed., 2005.

[2] A. Garcia-Piquer, "Knowledge Based Systems: Lecture Notes," La Salle - URL, Barcelona, 2023.

[3] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020.

[4] A. S. d. Garcez, L. C. Lamb, and D. M. Gabbay, *Neural-Symbolic Cognitive Reasoning*. Springer, 2008.

[5] N. J. Nilsson, *Principles of Artificial Intelligence*. Morgan Kaufmann, 2014.

[6] OpenAI, "GPT-4o System Card," 2024. [Online]. Available: https://openai.com/index/gpt-4o-system-card.
[7] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," *NeurIPS*, vol. 35, 2022.
[8] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008.
[9] Z. Liu et al., "DeepFashion: Powering Robust Clothes Recognition and Retrieval with Rich Annotations," in *Proceedings of IEEE CVPR*, 2016.
[10] R. He and J. McAuley, "Ups and downs: Modeling the visual evolution of fashion trends with one-class collaborative filtering," in *WWW*, 2016.
[11] W. L. Chao et al., "Forecasting fashion trends from street-style images," *arXiv preprint arXiv:1709.02325*, 2017.
[12] G. Adomavicius and A. Tuzhilin, "Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions," *IEEE Transactions on Knowledge and Data Engineering*, vol. 17, no. 6, 2005.
[13] B. Schwartz, *The Paradox of Choice: Why More Is Less*. New York: Harper Collins, 2004.
[14] Ellen MacArthur Foundation, "A New Textiles Economy: Redesigning fashion's future," 2017.
[15] K. Niinimäki et al., "The environmental price of fast fashion," *Nature Reviews Earth & Environment*, vol. 1, no. 4, pp. 189-200, 2020.
[16] M. Fowler and J. Lewis, "Microservices: a definition of this new architectural style," 2014. [Online].
[17] J. Han et al., "Survey on NoSQL database," in *6th International Conference on Pervasive Computing and Applications*, 2011.
[18] MongoDB Inc., "MongoDB Atlas Manual," [Online]. Available: https://www.mongodb.com/docs/atlas/.
[19] Streamlit Inc., "Streamlit Documentation," [Online]. Available: https://docs.streamlit.io.
[20] G. Van Rossum, *Python 3 Reference Manual*. CreateSpace, 2009.